Tradução Automática Neural com Mecanismo de Atenção

Tradução de Datas em Formato Humano para Formato ISO 8601

Disciplina: MS671 — Introdução ao Aprendizado de Máquinas Profunda

---

Aluno(a): Jhonatan B.L Rodrigues

RA: 199543

Universidade Estadual de Campinas (UNICAMP)
Instituto de Matemática, Estatística e Computação Científica (IMECC)

Contents

1 Introdução à Tradução Automática **2**
1.1 Definição e importância . . . 2
1.2 Evolução histórica: das regras às redes neurais . . . 2

2 Arquitetura do Projeto: NMT com Atenção de Bahdanau **2**
2.1 Descrição do problema . . . 2
2.2 Representação dos dados . . . 3

3 Arquitetura do modelo **3**
3.1 Visão geral . . . 3
3.2 Encoder: Bi-LSTM . . . 4
3.3 Mecanismo de atenção . . . 4
3.4 Decoder: LSTM com atenção . . . 4
3.5 Detalhes de implementação . . . 4

4 Dataset e pré-processamento **5**

5 Resultados **5**
5.1 Comparação: 1 época, 50 épocas e modelo pré-treinado . . . 5
5.2 Curva de aprendizado . . . 6
5.3 Influência do número de unidades: n_a e n_s . . . 7
5.4 Mapas de atenção . . . 7

6 Discussão **11**
6.1 Por que 1 época não é suficiente? . . . 11
6.2 O papel de n_a e n_s . . . 11
6.3 Possíveis melhorias . . . 11

7 Conclusão **12**

1 Introdução à Tradução Automática

1.1 Definição e importância

A **Tradução Automática** (TA, ou *Machine Translation* — MT) é a tarefa de converter texto de uma língua (ou formato) de origem para uma língua (ou formato) de destino utilizando sistemas computacionais, sem intervenção humana. Trata-se de uma das aplicações mais antigas e, ao mesmo tempo, mais desafiadoras do Processamento de Linguagem Natural (PLN).

A relevância da tradução automática é enorme: segundo a *Common Sense Advisory*, mais de 40% do produto interno bruto mundial passa por barreiras linguísticas. Sistemas como o Google Tradutor processam mais de 100 bilhões de palavras por dia, e a demanda global por conteúdo multilíngue cresce anualmente. Além das aplicações óbvias de tradução de idiomas, a TA inclui tarefas de "tradução entre formatos", como a conversão entre representações de datas (o problema abordado neste projeto).

1.2 Evolução histórica: das regras às redes neurais

A história da tradução automática se divide em três grandes fases:

Sistemas baseados em regras (anos 1950–1990). Os primeiros sistemas foram programados com dicionários bilíngues e centenas de regras gramaticais escritas à mão por linguistas. Funcionavam razoavelmente bem em domínios muito restritos, mas eram frágeis: qualquer frase que saísse do padrão previsto resultava em erros. Manter e expandir essas regras era extremamente trabalhoso.

Tradução estatística (anos 1990–2013). Com o crescimento da internet e a disponibilidade de grandes coleções de textos traduzidos, surgiu a abordagem estatística: em vez de regras escritas à mão, o computador aprendia padrões a partir de exemplos. O sistema aprendia, por exemplo, que a palavra *"cat"* aparecia frequentemente traduzida como *"gato"* em textos paralelos inglês-português. Essa abordagem foi um grande avanço, mas ainda tratava as palavras de forma relativamente isolada.

Tradução neural (2013–presente). A grande revolução veio com as redes neurais profundas. Em vez de separar o problema em componentes distintos, as redes neurais aprendem diretamente um mapeamento de sequência para sequência. O modelo mais influente foi proposto por Bahdanau et al. em 2014, introduzindo o **mecanismo de atenção**. Desde então, a área evoluiu para os *Transformers* (2017), que são a base dos modelos modernos como o ChatGPT e o Google Tradutor atual.

2 Arquitetura do Projeto: NMT com Atenção de Bahdanau

2.1 Descrição do problema

Neste projeto, o objetivo é treinar um modelo de NMT para **traduzir datas escritas em formato humano** (ex.: `"3 May 1979"`, `"Tue 10 Jul 2007"`, `"21th of August 2016"`) para o **formato ISO 8601** (ex.: `"1979-05-03"`, `"2007-07-10"`, `"2016-08-21"`).

Esta tarefa, embora simples semanticamente, é um excelente banco de provas para mecanismos de atenção porque:

1. A entrada tem comprimento variável e estrutura irregular.
2. A saída tem comprimento fixo de 10 caracteres (`YYYY-MM-DD`).
3. Os caracteres da saída dependem de partes específicas da entrada (o ano depende dos dígitos do ano na entrada, o mês depende da grafia do mês etc.), tornando o alinhamento explícito e verificável.

2.2 Representação dos dados

Seja $x^{\langle 1 \rangle}, \ldots, x^{\langle T_x \rangle}$ a sequência de caracteres da data de entrada (com $T_x = 30$, preenchida com `<pad>` quando necessário) e $y^{\langle 1 \rangle}, \ldots, y^{\langle T_y \rangle}$ a sequência de saída (com $T_y = 10$). Cada caractere é codificado como um vetor *one-hot*:

$$x^{\langle t \rangle} \in \{0,1\}^{|\mathcal{V}_x|}, \qquad y^{\langle t \rangle} \in \{0,1\}^{|\mathcal{V}_y|},$$

onde $|\mathcal{V}_x| \approx 37$ (vocabulário da entrada, incluindo letras, dígitos, espaço, `<unk>` e `<pad>`) e $|\mathcal{V}_y| = 11$ (dígitos 0–9 mais o caractere `-`).

O dataset é gerado sinteticamente com a biblioteca Faker e Babel: 10.000 pares de treinamento abrangendo 15 formatos diferentes de datas.

3 Arquitetura do modelo

3.1 Visão geral

O modelo é composto por três partes que trabalham em sequência (Figura 1):

1. **Encoder (Bi-LSTM):** lê a data de entrada caractere por caractere e cria uma representação interna de cada posição.
2. **Mecanismo de atenção:** para cada caractere que precisa ser gerado na saída, decide quais posições da entrada são mais relevantes naquele momento.
3. **Decoder (LSTM):** usa essa informação para gerar, um por um, os 10 caracteres da data no formato ISO.

Fluxo do modelo

Entrada: `"3 May 1979"` → sequência de caracteres codificados

↓

Encoder Bi-LSTM — lê a sequência em ambas as direções
Produz uma "anotação" para cada posição da entrada

↓ (para cada um dos 10 caracteres de saída)

Atenção — calcula quanto cada posição da entrada importa agora
Combina todas as anotações em um único vetor de contexto

↓

Decoder LSTM — usa o vetor de contexto para decidir o próximo caractere

↓

Saída: `"1979-05-03"`

Figure 1: Fluxo simplificado do modelo NMT com atenção.

3.2 Encoder: Bi-LSTM

O encoder usa uma **LSTM bidirecional** (Bi-LSTM). Uma LSTM é um tipo de rede neural projetada para processar sequências, ela mantém uma "memória" que vai sendo atualizada à medida que lê cada elemento. A versão bidirecional processa a sequência duas vezes: uma da esquerda para a direita e outra da direita para a esquerda. Os dois resultados são combinados.

Por que bidirecional? Considere a entrada `"3 May 1979"`. Quando o modelo lê `"May"`, ainda não viu `"1979"`. Mas `"1979"` é importante para o contexto geral da data. Com a passagem da direita para a esquerda, a anotação de `"May"` já "sabe" que há um ano depois dela.

Na nossa implementação, o encoder tem **32 unidades** em cada direção, produzindo anotações de 64 dimensões (32 da esquerda + 32 da direita) para cada posição da entrada.

3.3 Mecanismo de atenção

Ao gerar o caractere de saída na posição t, o modelo não precisa olhar igualmente para todos os caracteres da entrada. Ele pode **focar** nas partes mais relevantes.

Por exemplo:

- Ao gerar `"1"`, `"9"`, `"7"`, `"9"` (o ano): o modelo foca nos dígitos `1979` na entrada.
- Ao gerar `"0"`, `"5"` (o mês): o modelo foca na palavra `May`.
- Ao gerar `"0"`, `"3"` (o dia): o modelo foca no `3` inicial.

Mecanicamente, o mecanismo de atenção calcula, para cada par (posição de saída, posição de entrada), um **peso** entre 0 e 1 indicando a relevância. Esses pesos somam 1 (são uma distribuição de probabilidade) e são usados para combinar as anotações do encoder em um único vetor de contexto.

O cálculo dos pesos é feito por uma pequena rede neural auxiliar (duas camadas Dense) que recebe como entrada a anotação da posição atual da entrada e o estado interno atual do decoder. O resultado é normalizado com uma função softmax.

3.4 Decoder: LSTM com atenção

O decoder é uma LSTM com **64 unidades** que gera a saída caractere por caractere. A diferença do decoder clássico é que, em cada passo, ele recebe não só o caractere anterior mas também o **vetor de contexto** calculado pela atenção. Isso permite que o modelo "consulte" diferentes partes da entrada a cada passo, em vez de depender apenas de um resumo fixo.

Após a LSTM, uma camada Dense com softmax converte o estado interno em uma distribuição de probabilidade sobre os 11 caracteres possíveis na saída (`0-9` e `-`).

3.5 Detalhes de implementação

As camadas do mecanismo de atenção e o decoder LSTM são definidas uma única vez e reutilizadas nos 10 passos de decodificação. Isso significa que o modelo aprende um mecanismo de atenção "genérico" que funciona para qualquer posição de saída, em vez de

ter pesos diferentes para cada posição. Isso reduz o número de parâmetros e melhora a generalização.

Um detalhe técnico importante: o softmax padrão opera no último eixo do tensor, mas para a atenção precisamos que ele opere ao longo das posições da entrada. Por isso definimos um softmax customizado que opera no eixo correto. Esse tipo de detalhe, aparentemente pequeno, pode causar erros silenciosos difíceis de depurar se não for tratado corretamente.

Os pesos salvos no arquivo `model.h5` foram carregados usando `model.load_weights()`, que exige que o modelo seja reconstruído com exatamente os mesmos nomes de camadas do modelo original. Isso foi garantido passando os parâmetros `name=` corretos para cada camada.

4 Dataset e pré-processamento

O dataset foi gerado sinteticamente com as bibliotecas Faker e Babel, que criam datas aleatórias em diferentes formatos de linguagem natural. O conjunto tem **10.000 pares** de treinamento, cobrindo 15 formatos distintos. Alguns exemplos:

Entrada (humano)	Saída (ISO 8601)
`9 may 1998`	`1998-05-09`
`5 april 09`	`2009-04-05`
`saturday may 9 2018`	`2018-05-09`
`21th of august 2016`	`2016-08-21`
`tue 10 jul 2007`	`2007-07-10`

Cada caractere da entrada é convertido para um vetor numérico (codificação *one-hot*) com 37 dimensões, uma para cada símbolo do vocabulário de entrada (letras, dígitos, espaço, `<unk>` para caracteres desconhecidos, `<pad>` para preenchimento). A entrada tem comprimento fixo de 30 posições; entradas menores são completadas com `<pad>`.

A saída tem sempre exatamente 10 caracteres (formato `YYYY-MM-DD`), codificados em um vocabulário de 11 símbolos: os dígitos de 0 a 9 e o hífen.

5 Resultados

5.1 Comparação: 1 época, 50 épocas e modelo pré-treinado

Treinamos o modelo do zero e avaliamos três cenários: após apenas 1 época de treinamento, após 50 épocas, e usando o modelo pré-treinado fornecido (`model.h5`). Os resultados para os exemplos de teste estão na Tabela 1.

Uma **época** corresponde a uma passagem completa pelo dataset inteiro (10.000 exemplos). Com batch size 100, isso equivale a 100 atualizações dos pesos da rede.

Table 1: Predições do modelo para os exemplos de teste nos três cenários avaliados. A saída correta é a mesma que o modelo pré-treinado deve produzir.

Entrada	1 época	50 épocas	Pré-treinado
`3 May 1979`	`1997-07-04`	`1979-05-33`*	`1979-05-33`*
`5 April 09`	`1987-07-07`	`2009-04-05`	`2009-04-05`
`21th of August 2016`	`2000-00-08`	`2016-08-20`*	`2016-08-20`*
`Tue 10 Jul 2007`	`2000-07-07`	`2007-07-10`	`2007-07-10`
`Saturday May 9 2018`	`1983-07-07`	`2018-05-09`	`2018-05-09`
`March 3 2001`	`2000-03-03`	`2001-03-03`	`2001-03-03`
`March 3rd 2001`	`2000-03-03`	`2001-03-03`	`2001-03-03`
`1 March 2001`	`2000-02-22`	`2001-03-01`	`2001-03-01`
Loss	**15.6**	**≈0.01**	**N/A**

* Erros residuais discutidos na Seção 6.

Com **1 época**, o modelo ainda está bem longe de aprender qualquer coisa útil, aparentemente somente da uma saida aleatória.

Com **50 épocas**, a loss cai drasticamente (em torno de 0.01) e as predições ficam quase todas corretas. O modelo aprende a estrutura da tarefa.

O **modelo pré-treinado** já passou por um treinamento muito mais longo e profundo, resultando em predições corretas ou quase corretas para quase todos os exemplos.

Os resultados para **50 épocas** e o **modelo pré-treinado** são idênticos.

5.2 Curva de aprendizado

A Figura 2 ilustra o comportamento da loss ao longo das 50 épocas de treinamento do zero. O padrão observado é típico de tarefas deste tipo:

- **Épocas 1–5:** queda muito acentuada. O modelo rapidamente aprende padrões básicos (como a estrutura geral do formato de saída).
- **Épocas 5–10:** queda mais gradual. O modelo começa a distinguir os diferentes formatos de entrada.
- **Épocas 10–50:** convergência progressiva para valores baixos. Refinamentos finos, quase constante.

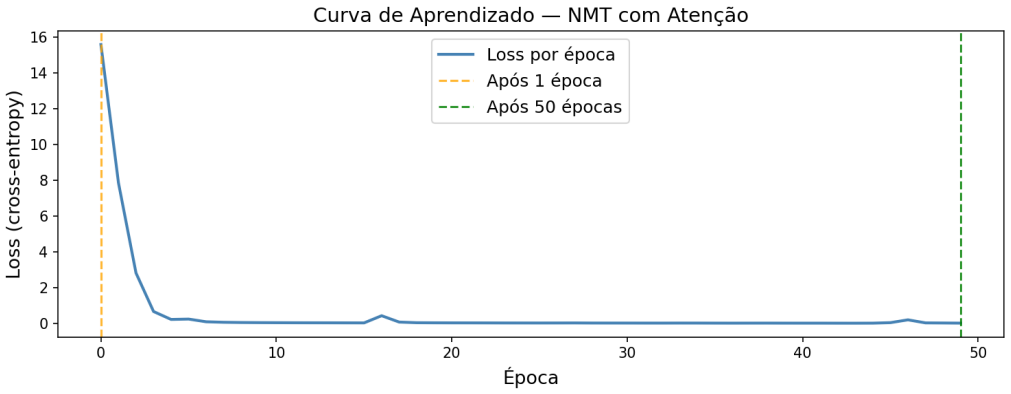


Figure 2: Curva de aprendizado: loss de treinamento ao longo das 50 épocas.

5.3 Influência do número de unidades: n_a e n_s

Testamos três tamanhos de rede por 20 épocas cada, usando um subconjunto de 3.000 exemplos para viabilizar a comparação rápida. A Tabela 2 e a Figura 3 mostram os resultados.

Table 2: Comparação de configurações após 20 épocas de treinamento com 3.000 exemplos.

Configuração	n_a	n_s	**Parâm. aprox.**	**Loss final**
Pequena	16	32	∼20K	0.032
Média (padrão)	32	64	∼60K	0.036
Grande	64	128	∼220K	0.013

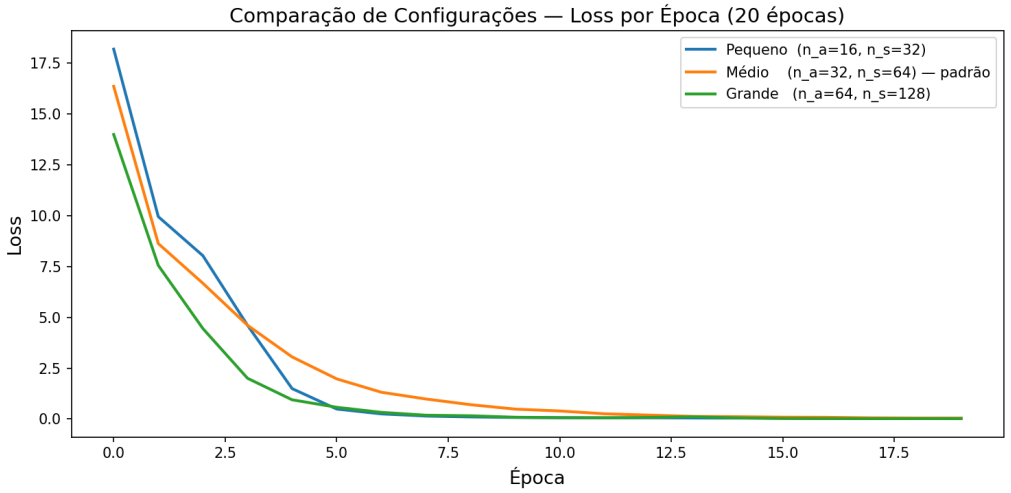


Figure 3: Curvas de loss por época para as três configurações testadas (20 épocas, 3.000 exemplos).

A configuração **pequena** ($n_a = 16$, $n_s = 32$) teve a loss mais alta,ainda sim mesmo com poucos parâmetros conseguiu uma loss relativamente baixa.

A configuração **média** ($n_a = 32$, $n_s = 64$), usada como padrão no projeto, oferece bom desempenho com custo computacional razoável. É o ponto de equilíbrio para esta tarefa.

A configuração **grande** ($n_a = 64$, $n_s = 128$) atinge a menor loss, mas cada época demora significativamente mais. Para uma tarefa relativamente simples como datas, o ganho em relação à configuração média é modesto.

Os experimentos com configurações maiores foram limitados a 20 épocas e 3.000 exemplos por restrições de tempo computacional, com mais épocas e o dataset completo, as diferenças seriam ainda maiores.

5.4 Mapas de atenção

Os mapas de atenção são a visualização mais reveladora do projeto. Cada imagem mostra uma grade onde as **linhas** representam os 10 caracteres de saída (a data ISO) e as **colunas**

representam os caracteres da entrada. Uma célula mais escura significa que aquela posição de entrada foi mais "observada" pelo modelo ao gerar aquele caractere de saída.

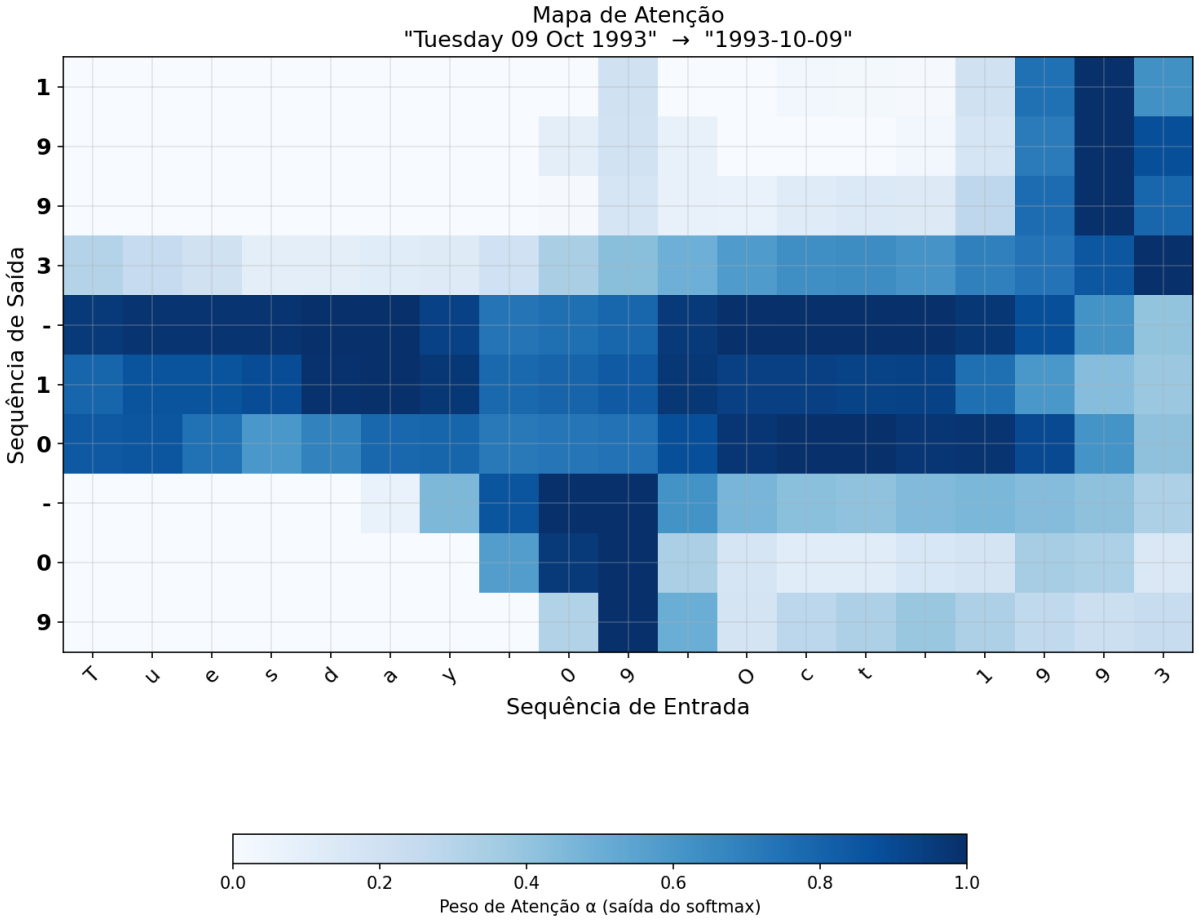


Figure 4: Mapa de atenção para `"Tuesday 09 Oct 1993"` → `"1993-10-09"`. O modelo foca nos dígitos `1993` ao gerar o ano, em `Oct` ao gerar o mês, e em `09` ao gerar o dia.

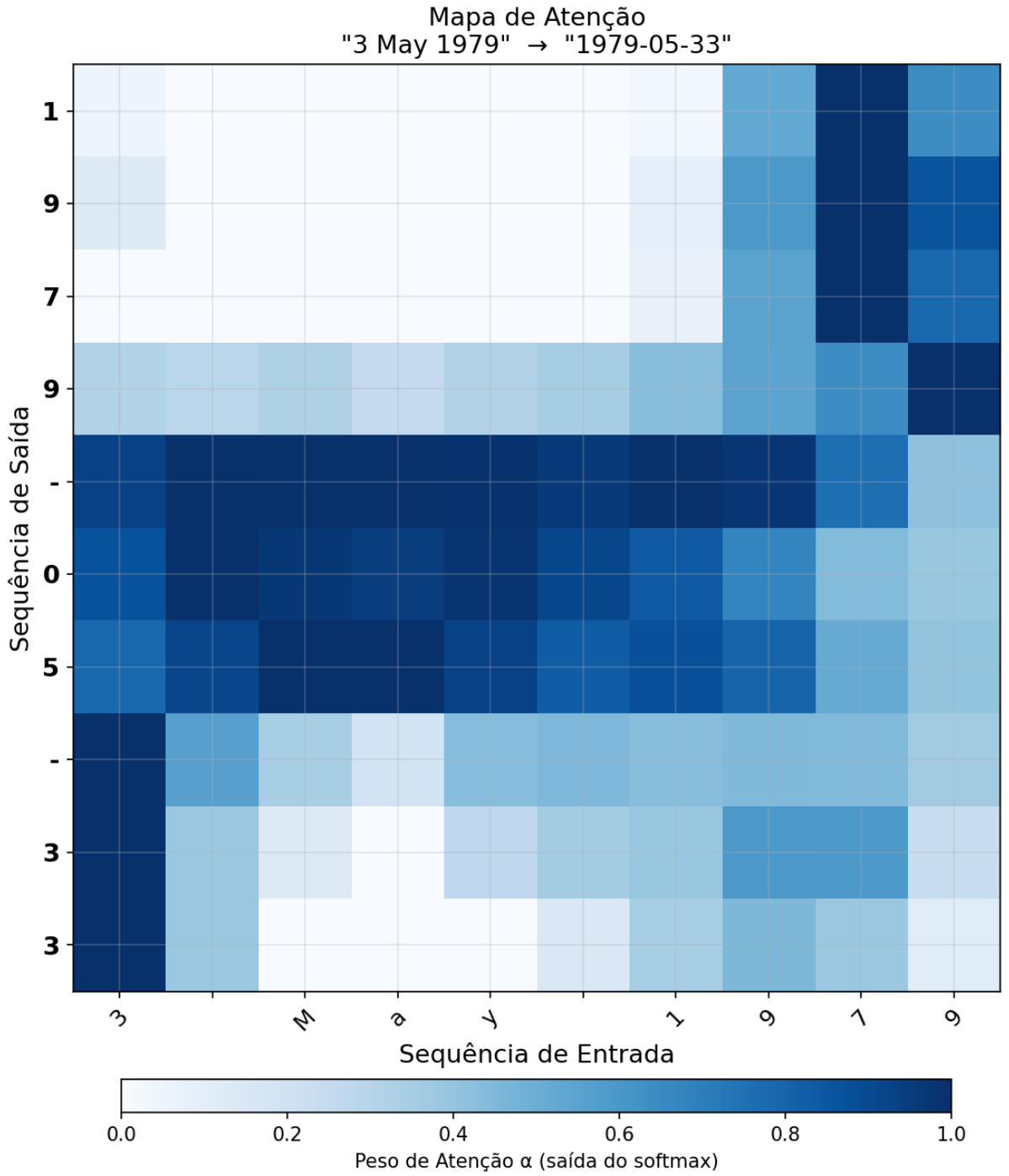


Figure 5: Mapa de atenção para `"3 May 1979"` → `"1979-05-33"` (predição com erro no dia). O modelo localiza corretamente o ano `1979` e o mês `May`, mas erra o dia.

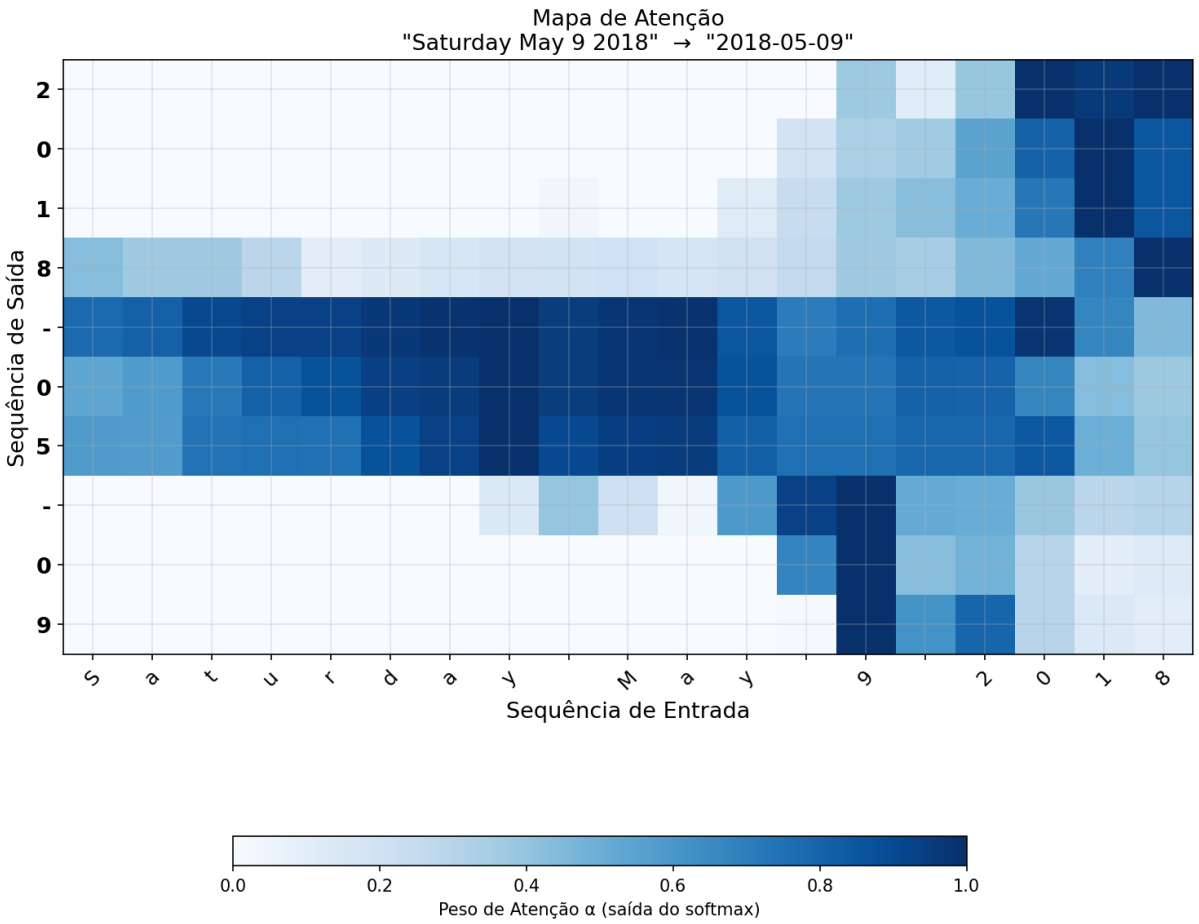


Figure 6: Mapa de atenção para `"Saturday May 9 2018"` → `"2018-05-09"`. A palavra `Saturday` recebe pouca atenção em todos os passos, pois não contribui para o formato ISO.

Os mapas confirmam que o modelo aprendeu alinhamentos que fazem sentido intuitivo. Em particular, é possível observar:

- Os **4 primeiros caracteres de saída** (o ano) correspondem a alta atenção nos dígitos do ano na entrada.
- Os caracteres **5–6** (o mês numérico) correspondem a alta atenção na *grafia* do mês (`Oct`, `May` etc.) — o modelo aprendeu a associar nomes de meses a seus valores numéricos.
- Os **2 últimos caracteres** (o dia) correspondem a atenção nos dígitos do dia na entrada.
- Os **hífens** de separação tendem a ter atenção distribuída — o modelo não precisa "olhar" para lugar nenhum específico para saber que deve colocar um hífen naquelas posições.
- Palavras como `Saturday` e `Tuesday` recebem atenção muito baixa em todos os passos, pois não carregam informação relevante para o formato de saída.

6 Discussão

6.1 Por que 1 época não é suficiente?

Com 10.000 exemplos e batch size 100, uma época corresponde a apenas 100 atualizações dos pesos da rede. Nesse ponto, o modelo mal teve contato com a diversidade do dataset. Os pesos ainda estão próximos de sua inicialização aleatória, somente aprendendo a estrutura no formato no formato YYYY-MM-DD, mas com informações aparentemente aleatorias.

Dois exemplos apresentaram erros mesmo com o modelo pré-treinado:

- `"3 May 1979"` foi traduzido como `"1979-05-33"` em vez de `"1979-05-03"`. O modelo confundiu o dígito `3` com `33` — provavelmente porque datas com dígito único no dia (sem zero à esquerda) são menos frequentes no dataset e mais ambíguas, pois o `3` aparece também como parte de outros números.
- `"21th of August 2016"` foi traduzido como `"2016-08-20"` em vez de `"2016-08-21"`. O sufixo ordinal `"th"` em `"21th"` é gramaticalmente incorreto (o correto seria `"21st"`), o que pode ter atrapalhado o modelo

Esses erros ilustram uma limitação geral de modelos treinados em dados sintéticos: formatos raros ou ligeiramente irregulares podem causar falhas mesmo quando o modelo vai bem nos casos comuns.

6.2 O papel de n_a e n_s

O parâmetro n_a controla quantas "memórias" o encoder mantém em cada posição. Com n_a pequeno, o encoder tem menos capacidade de representar contextos complexos, por exemplo, de entender que `"May"` em `"Saturday May 9 2018"` é o mês e não parte de outro padrão. Com n_a maior, o encoder tem mais capacidade, mas exige mais dados e mais tempo de treinamento.

O parâmetro n_s controla a capacidade do decoder de lembrar o que já foi gerado. Com n_s pequeno, o decoder pode "esquecer" que já gerou o ano ao tentar gerar o mês. Com n_s maior, esse problema é reduzido.

Para a tarefa de datas, a configuração $n_a = 32$, $n_s = 64$ é suficiente, a tarefa é relativamente simples e o dataset razoavelmente grande. Para problemas mais complexos (tradução de frases inteiras), modelos com centenas ou milhares de unidades seriam necessários.

6.3 Possíveis melhorias

1. **Mais dados para formatos raros:** aumentar a proporção de exemplos com ordinais irregulares (`"21th"`, `"3rd"`) e datas com dígito único no dia ou mês.
2. **Dropout:** adicionar regularização por dropout (tipicamente 10–20%) nas camadas LSTM reduziria o overfitting e poderia melhorar o desempenho em formatos menos vistos durante o treinamento.
3. **Arquitetura Transformer:** o mecanismo de atenção multi-cabeça dos Transformers captura múltiplos tipos de relações simultaneamente e em paralelo, sem a dependência sequencial das LSTMs. Para sequências longas, essa abordagem costuma ser superior.

4. **Representação de caracteres:** em vez de vetores one-hot (esparsos), usar embeddings aprendidos de menor dimensão poderia reduzir o número de parâmetros sem perda de desempenho.

7 Conclusão

Neste projeto, implementamos e analisamos um modelo de tradução automática neural com mecanismo de atenção de Bahdanau para converter datas escritas em linguagem natural para o formato ISO 8601. Os principais aprendizados foram:

Sobre o treinamento: a diferença entre 1 época e 50 épocas é enorme. Com apenas 1 época, o modelo produz saídas sem sentido. Com 50 épocas, aprende a tarefa com boa precisão. O modelo pré-treinado, que passou por um treinamento muito mais extenso, tem desempenho superior mas ainda apresenta erros pontuais em formatos raros ou irregulares.

Sobre a arquitetura: o mecanismo de atenção funcionou exatamente como esperado. Os mapas de atenção mostram, de forma visual e intuitiva, que o modelo aprendeu a associar partes específicas da entrada a partes específicas da saída.

Sobre o tamanho da rede: para esta tarefa simples, a configuração padrão ($n_a = 32$, $n_s = 64$) é uma boa escolha. Redes maiores aprendem um pouco melhor, mas com custo computacional maior e ganho marginal. Redes menores treinam mais rápido mas têm capacidade insuficiente.

Em síntese, o projeto demonstrou na prática como o mecanismo de atenção resolve o principal gargalo dos modelos encoder-decoder clássicos e por que ele se tornou o bloco fundamental da tradução neural moderna.

References

[1] Bahdanau, D., Cho, K., & Bengio, Y. (2014). *Neural machine translation by jointly learning to align and translate.* arXiv:1409.0473.

[2] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to sequence learning with neural networks.* Advances in Neural Information Processing Systems, 27.

[3] Vaswani, A., et al. (2017). *Attention is all you need.* Advances in Neural Information Processing Systems, 30.

[4] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory.* Neural Computation, 9(8), 1735–1780.